

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ
по дисциплине «Введение в нереляционные базы данных»
Тема: Каталог ВУЗов

Студенты гр. 3342

Иванов Д.М.

Корниенко А.Е.

Русанов А.И.

Преподаватель

Заславский М.М

Санкт-Петербург

2026

ЗАДАНИЕ

Студенты

А.Е. Корниенко

А.И. Русанов

Д.М. Иванов

Группа 3342

Тема проекта: Разработка приложения для управления библиотечными карточками.

Исходные данные:

Необходимо разработать приложение “Каталог ВУЗов для поступающих - специальности, факультеты, условия, баллы, калькуляторы поступления и аналитика” на MongoDB.

Содержание пояснительной записки:

«Содержание»

«Введение»

«Качественные требования к решению»

«Сценарий использования»

«Модель данных»

«Разработка приложения»

«Вывод»

«Приложение»

Предполагаемый объем пояснительной записки:

Не менее 10 страниц.

Дата выдачи задания:

Дата сдачи реферата:

Дата защиты реферата:

Студенты гр. 3342		Иванов Д.М.
		Корниенко А.Е.
		Русанов А.И.
Преподаватель		Заславский М.М.

АННОТАЦИЯ

В рамках данного курса предполагалось разработать приложение на одну из заданных тем, использующее нереляционную базу данных, в команде из трёх человек. Была выбрана тема создания приложения “Каталог ВУЗов для поступающих - специальности, факультеты, условия, баллы, калькуляторы поступления и аналитика”. Для хранения данных использовалась MongoDB. Найти исходный код и всю дополнительную информацию можно по ссылке: <https://github.com/moevm/nsql1h26-uni>

SUMMARY

This course involved developing an application on one of the given topics using a non-relational database in a team of three. The chosen project topic was the development of a university directory application for prospective students, featuring majors, faculties, admission requirements, score thresholds, admission calculators, and analytics. MongoDB was used for data storage. The source code and all additional information can be found at the following link: <https://github.com/moevm/nsql1h26-uni>

СОДЕРЖАНИЕ

Введение	6
1. Сценарии использования	8
1.1. Макеты UI	8
1.2. Сценарии для неавторизованного пользователя	9
1.3. Сценарии для авторизованного пользователя	12
1.4. Сценарии, которые будут представлены в рамках прототипа “Хранение и представление данных”	14
2. Модель данных	16
2.1. Нереляционная модель	16
2.2. Реляционная модель	32
2.3. Сравнение моделей	42
3. Разработанное приложение	45
3.1. Краткое описание и технологический стек	45
3.2. Снимки экрана приложения	46
4 Выводы	48
5 Приложение	49
6 Литература	50

ВВЕДЕНИЕ

Актуальность решаемой проблемы.

Сегодня данные о специальностях, факультетах, проходных баллах и условиях поступления в вузы разрознены: они находятся на десятках неудобных официальных сайтов, в PDF-файлах приказов, на форумах и в таблицах, которые не связаны между собой. Абитуриентам и их родителям приходится тратить часы на поиск и сравнение информации, что создаёт высокий порог входа в процесс выбора учебного заведения. Ситуация усугубляется тем, что данные ежегодно меняются: обновляются проходные баллы, количество бюджетных мест, стоимость контракта и перечень вступительных испытаний. Кроме того, структура представления информации в разных вузах не унифицирована — у одного факультет делится на кафедры, у другого сразу перечислены специальности, что затрудняет автоматизированное сравнение. В период подачи документов (июнь–август) нагрузка на информационные системы возрастает, а отсутствие удобного инструмента для расчёта вероятности поступления на основе индивидуальных баллов ЕГЭ и льгот вызывает у абитуриентов повышенную тревожность. Наконец, без единой платформы невозможно строить персонализированные рекомендации, которые помогли бы поступающему найти специальности, соответствующие его профилю, даже если он о них изначально не знал.

Постановка задачи.

Необходимо реализовать приложение для работы с базой данных для каталога ВУЗов.

Предлагаемое решение.

Нужно реализовать веб-приложение, использующее MongoDB. Приложение позволит хранить карточки актёров и удобно с ними взаимодействовать.

Качественные требования к решению.

Должен поддерживаться массовый импорт/экспорт. Требуется функция фильтрации данных, поиска вузов, направлений, необходимых баллов, добавления новых сущностей.

1. СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

1.1. Макеты UI



Рисунок 1 - все экраны приложения

Макет в репозитории:

<https://github.com/moevm/nsql1h26-uni/blob/main/docs/images/ui-mockup.png>

1.2. Сценарии для неавторизованного пользователя

Сценарий 1: Поиск вузов по названию

Действующее лицо: Неавторизованный пользователь / абитуриент

Предварительные условия: Пользователь находится на главной странице.

Основной сценарий:

1. Пользователь вводит название или начало названия интересующего вуза в поисковую строку.
2. Нажимает на кнопку поиска.
3. Приложение отображает на странице блоки, в каждом из которых содержится название вуза и некоторая дополнительная информация.

Сценарий 2: Поиск вузов по фильтрам

Действующее лицо: Неавторизованный пользователь / абитуриент

Предварительные условия: Пользователь находится на главной странице.

Основной сценарий:

1. Пользователь выбирает в разделе с фильтрами необходимую ему информацию.
2. Нажимает на кнопку поиска.
3. Приложение отображает на странице блоки, в каждом из которых содержится название вуза и некоторая информация.

Сценарий 3: Просмотр страницы определенного вуза

Действующее лицо: Неавторизованный пользователь / абитуриент

Предварительные условия: Пользователь находится на главной странице.

Отображены блоки с вузами.

Основной сценарий:

1. Пользователь нажимает на блок выбранного вуза.
2. Приложение переходит на новую страницу с детальной информацией по выбранному вузу.

Сценарий 4: Поиск направлений вуза по названию

Действующее лицо: Неавторизованный пользователь / абитуриент

Предварительные условия: Пользователь находится на странице определенного вуза.

Основной сценарий:

1. Пользователь вводит название или начало названия интересующего направления в поисковую строку.
2. Нажимает на кнопку поиска.
3. Приложение отображает на странице блоки, в каждом из которых содержится название направления и некоторая дополнительная информация.

Сценарий 5: Поиск направлений вуза по фильтрам

Действующее лицо: Неавторизованный пользователь / абитуриент

Предварительные условия: Пользователь находится на странице определенного вуза.

Основной сценарий:

1. Пользователь выбирает в разделе с фильтрами необходимую ему информацию.
2. Нажимает на кнопку поиска.
3. Приложение отображает на странице блоки, в каждом из которых содержится название направления и остальная по нему информация.

Сценарий 6: Использование калькулятора ЕГЭ

Действующее лицо: Неавторизованный пользователь / абитуриент

Предварительные условия: Пользователь находится на странице с калькулятором ЕГЭ.

Основной сценарий:

1. Пользователь выбирает интересные ему предметы ЕГЭ, выставляет в них баллы ЕГЭ.
2. Нажимает на кнопку поиска.
3. Приложение отображает на странице те направления из разных вузов, в которые мог бы поступить абитуриент с выбранными баллами.

Сценарий 7: Авторизация в приложении

Действующее лицо: Неавторизованный пользователь

Предварительные условия: Пользователь находится на странице авторизации.

Основной сценарий:

1. Пользователь вводит логин и пароль в поля ввода.
2. Нажимает на кнопку входа.
3. Если данный логин есть в системе и пароли совпадают, то идет переход на главную страницу и пользователь начинает работать как администратор.

В противном случае выводится информация о неверных данных, и пользователь остается на той же странице.

Сценарий 8: Просмотр статистики

Действующее лицо: Неавторизованный пользователь / абитуриент

Предварительные условия: Пользователь находится на странице статистики.

Основной сценарий:

1. Пользователь выбирает атрибуты для осей X, Y.
2. Пользователь задает диапазоны или варианты для каждого атрибута.
3. Нажимает на кнопку построения диаграммы.

4. На странице отображается кастомизируемая статистика в виде диаграммы.

1.3. Сценарии для авторизованного пользователя

Сценарий 9: Добавление нового вуза

Действующее лицо: Авторизованный пользователь

Предварительные условия: Администратор находится в страница администратора.

Основной сценарий:

1. Администратор нажимает на кнопку добавления вуза.
2. Открывается модальное окно.
3. Администратор вводит в окне всю необходимую информацию о новом вузе.
4. Нажимает на кнопку сохранения.
5. Если все обязательные поля заполнены, база данных обновляется, администратор переходит на главную страницу.

В противном случае выводится информация о неверных данных, и пользователь остается в окне.

Сценарий 10: Добавление нового направления

Действующее лицо: Авторизованный пользователь

Предварительные условия: Администратор находится страница администратора.

Основной сценарий:

1. Администратор нажимает на кнопку добавления направления.
2. Открывается модальное окно.
3. Администратор вводит в окне необходимый вуз и всю информацию о новом направлении.
4. Нажимает на кнопку сохранения.
5. Если все обязательные поля заполнены, база данных обновляется, администратор переходит на главную страницу.

В противном случае выводится информация о неверных данных, и пользователь остается в окне.

Сценарий 11: Редактирование вуза

Действующее лицо: Авторизованный пользователь

Предварительные условия: Администратор находится на страница администратора. Отображены блоки с вузами.

Основной сценарий:

1. Администратор нажимает на кнопку редактирования вуза, находящуюся в блоке определенного вуза.
2. Открывается окно редактирования с заполненными текущими данными о вузе.
3. Администратор вводит обновленные данные.
4. Нажимает на кнопку сохранения.
5. Если все обязательные поля заполнены, база данных обновляется, администратор переходит на главную страницу.

В противном случае выводится информация о неверных данных, и пользователь остается в окне.

Сценарий 12: Редактирование направления

Действующее лицо: Авторизованный пользователь

Предварительные условия: Администратор находится на страница администратора. Отображены блоки с направлениями.

Основной сценарий:

1. Администратор нажимает на кнопку редактирования, находящуюся в блоке определенного направления.
2. Открывается окно редактирования направления с заполненными текущими данными о направлении.
3. Администратор вводит обновленные данные.
4. Нажимает на кнопку сохранения.

5. Если все обязательные поля заполнены, база данных обновляется, администратор переходит на главную страницу.

В противном случае выводится информация о неверных данных, и пользователь остается в окне.

Сценарий 13: Массовый экспорт данных

Действующее лицо: Авторизованный пользователь

Предварительные условия: Администратор находится на главной странице. Отображены блоки с вузами.

Основной сценарий:

1. Администратор нажимает на кнопку экспорта.
2. Открывается директория локального компьютера, в которой выбирается место для сохранения файла в формате JSON.
3. После нажатия на кнопку подтверждения администратор остается на той же странице. Получает сообщение об успешности сохранения файла.

Сценарий 14: Массовый импорт данных

Действующее лицо: Авторизованный пользователь

Предварительные условия: Администратор находится на главной странице.

Основной сценарий:

1. Администратор нажимает на кнопку импорта.
2. Открывается директория локального компьютера, в которой выбирается необходимый файл.
3. После нажатия на кнопку подтверждения администратор остается на той же странице. Получает сообщение об успешности сохранения информации в базе данных.

1.4. Сценарии, которые будут представлены в рамках прототипа “Хранение и представление данных”

- Сценарий 1: Поиск вузов по названию
- Сценарий 2: Поиск вузов по фильтрам
- Сценарий 4: Поиск направлений вуза по названию
- Сценарий 5: Поиск направлений вуза по фильтрам
- Сценарий 7: Авторизация в приложении
- Сценарий 9: Добавление нового вуза
- Сценарий 10: Добавление нового направления

Вывод: для реализованного решения в рамках прототипа «Хранение и представление данных» будут преобладать операции чтения и фильтрации данных, поскольку основная ценность приложения для неавторизованного пользователя — это быстрый поиск вузов и направлений как по названию, так и по набору фильтров, а также удобное отображение найденной информации. При этом уже присутствуют и операции создания (записи) новых сущностей — добавление вузов и направлений со стороны администратора, что необходимо для наполнения и актуализации каталога. Операции редактирования, импорта/экспорта и работы со статистикой вынесены за рамки данного прототипа, так как они требуют дополнительной проработки интерфейса и логики обработки данных. Таким образом, в рамках прототипа основная нагрузка ложится на чтение и фильтрацию, а запись носит вспомогательный, но необходимый характер для поддержания базы в актуальном состоянии.

2. МОДЕЛЬ ДАННЫХ

2.1. Нереляционная модель данных

Графическое представление

universities		programs		admins	
_id: ObjectId	12 bytes	_id: ObjectId	12 bytes	_id: ObjectId	12 bytes
name: string	255 bytes	university_id: ObjectId	12 bytes	username: string	50 bytes
city: string	100 bytes	name: string	255 bytes	password_hash: string	60 bytes
has_dormitory: bool	1 byte	budget_places: int	4 bytes	createdAt: date	8 bytes
military_dept: bool	1 byte	passing_score: int	4 bytes		
website: string	255 bytes	comment: string	1024 bytes		
comment: string	1024 bytes	createdAt: date	8 bytes		
createdAt: date	8 bytes	updatedAt: date	8 bytes		
updatedAt: date	8 bytes	required_subjects: [{	<array>		
		subject: string	50 bytes		
		minimum_points: int	4 bytes		
		}]			
		Total:	54bytes/el		

Рисунок 2 – Нереляционная модель

```
``json
```

```
[
{
  "collection": "universities",
  "description": "Коллекция вузов",
  "fields": [
    {
      "name": "_id",
      "bsonType": "objectId",
      "description": "Уникальный идентификатор вуза",
      "required": true
    },
    {
      "name": "name",
      "bsonType": "string",
      "description": "Полное название вуза",
      "required": true
    }
  ]
}
```



```
},  
{  
  "name": "city",  
  "bsonType": "string",  
  "description": "Город",  
  "required": true  
},  
{  
  "name": "has_dormitory",  
  "bsonType": "bool",  
  "description": "Наличие общежития",  
  "required": true  
},  
{  
  "name": "military_dept",  
  "bsonType": "bool",  
  "description": "Наличие военной кафедры",  
  "required": true  
},  
{  
  "name": "website",  
  "bsonType": "string",  
  "description": "Сайт вуза",  
  "required": false  
},  
{  
  "name": "comment",  
  "bsonType": "string",  
  "description": "Произвольный комментарий",  
  "required": false
```

```

    },
    {
      "name": "createdAt",
      "bsonType": "timestamp",
      "required": true,
      "description": "Дата создания записи"
    },
    {
      "name": "updatedAt",
      "bsonType": "timestamp",
      "required": true,
      "description": "Дата обновления записи"
    }
  ]
},
{
  "collection": "programs",
  "description": "Коллекция направлений подготовки",
  "fields": [
    {
      "name": "_id",
      "bsonType": "objectId",
      "description": "Уникальный идентификатор программы",
      "required": true
    },
    {
      "name": "university_id",
      "bsonType": "objectId",
      "description": "Ссылка на вуз",
      "required": true
    }
  ]
}

```

```

    },
    {
      "name": "name",
      "bsonType": "string",
      "description": "Название направления",
      "required": true
    },
    {
      "name": "budget_places",
      "bsonType": "int",
      "description": "Количество бюджетных мест",
      "required": true
    },
    {
      "name": "passing_score",
      "bsonType": "int",
      "description": "Проходной балл прошлого года",
      "required": true
    },
    {
      "name": "required_subjects",
      "bsonType": "array",
      "description": "Список необходимых предметов ЕГЭ",
      "items": {
        "bsonType": "object",
        "fields": [
          {
            "name": "subject",
            "bsonType": "string",
            "description": "Название предмета",

```

```

        "required": true
    },
    {
        "name": "minimum_points",
        "bsonType": "int",
        "description": "Минимальное количество баллов",
        "required": true
    }
]
}
},
{
    "name": "comment",
    "bsonType": "string",
    "description": "Произвольный комментарий",
    "required": false
},
{
    "name": "createdAt",
    "bsonType": "timestamp",
    "required": true,
    "description": "Дата создания записи"
},
{
    "name": "updatedAt",
    "bsonType": "timestamp",
    "required": true,
    "description": "Дата обновления записи"
}
]

```

```

},
{
  "collection": "admins",
  "description": "Администраторы",
  "fields": [
    {
      "name": "_id",
      "bsonType": "objectId",
      "description": "Уникальный идентификатор администратора",
      "required": true
    },
    {
      "name": "username",
      "bsonType": "string",
      "required": true,
      "description": "Логин администратора"
    },
    {
      "name": "password_hash",
      "bsonType": "string",
      "required": true,
      "description": "Хэш пароля"
    },
    {
      "name": "createdAt",
      "bsonType": "timestamp",
      "required": true,
      "description": "Дата создания записи"
    }
  ]
}

```

```
}  
]  
...
```

Описание назначений коллекций, типов данных и сущностей

В модели описаны три коллекции: universities, programs и admins.

Коллекция universities

Коллекция содержит документы с информацией о каждом вузе:

- ``_id`` — уникальный идентификатор вуза (встроенный тип ObjectId, занимает 12 байт);
- ``name`` — полное название вуза (строка, до 255 байт);
- ``city`` — город, в котором находится вуз (строка, до 100 байт);
- ``has_dormitory`` — наличие общежития (логический тип, 1 байт);
- ``military_dept`` — наличие военной кафедры (логический тип, 1 байт);
- ``website`` — ссылка на официальный сайт вуза (строка, до 255 байт, необязательное поле);
- ``comment`` — произвольный комментарий (строка, до 1024 байт, необязательное поле);
- ``createdAt`` — дата и время создания записи (timestamp, 8 байт);
- ``updatedAt`` — дата и время последнего обновления записи (timestamp, 8 байт).

Коллекция programs

Коллекция хранит информацию о направлениях подготовки (программах) в вузах:

- ``_id`` — уникальный идентификатор программы (ObjectId, 12 байт);
- ``university_id`` — ссылка на вуз, в котором реализуется программа (ObjectId, 12 байт);
- ``name`` — название направления подготовки (строка, до 255 байт);

- ``budget_places`` — количество бюджетных мест (целое число, 4 байта);
- ``passing_score`` — проходной балл прошлого года (целое число, 4 байта);
- ``required_subjects`` — массив объектов с информацией о необходимых предметах ЕГЭ. Каждый элемент массива содержит:
 - ``subject`` — название предмета (строка, до 50 байт);
 - ``minimum_points`` — минимальное количество баллов (целое число, 4 байта);
- ``comment`` — произвольный комментарий (строка, до 1024 байт, необязательное поле);
- ``createdAt`` — дата создания записи (timestamp, 8 байт);
- ``updatedAt`` — дата обновления записи (timestamp, 8 байт).

Коллекция admins

Коллекция хранит информацию об администраторах платформы:

- ``_id`` — уникальный идентификатор администратора (ObjectId, 12 байт);
- ``username`` — логин администратора (строка, до 50 байт);
- ``password_hash`` — хэш пароля (строка, 60 байт для bcrypt);
- ``createdAt`` — дата создания записи (timestamp, 8 байт).

Избыточность данных

В модели существует потенциальная избыточность, связанная с хранением названий предметов ЕГЭ непосредственно в каждом документе направления подготовки. Общий коэффициент избыточности вычисляется как отношение фактического объёма модели к "чистому" объёму (объёму без дублирования).

Введем следующие переменные:

- U — количество вузов;
- K_dir — среднее количество направлений на один вуз (на основе анализа предметной области);
- $K_subjects$ — среднее количество предметов ЕГЭ на одно направление (принято равным 3);
- A — количество администраторов (константа, равна 3).

Расчет "чистого" объема данных

Для устранения избыточности можно было бы вынести предметы ЕГЭ в отдельную коллекцию, но в текущей реализации они хранятся внутри документов `programs`. Оценим, какой объем занимали бы данные при нормализованном подходе.

Для "чистой" модели (с вынесенными предметами):

- Создается отдельная коллекция `subjects` со списком всех возможных предметов ЕГЭ;
- В коллекции `programs` вместо массива объектов хранится массив элементов, каждый из которых содержит:
 - идентификатор предмета (`ObjectId`, 12 байт);
 - минимальное количество баллов (`int`, 4 байта).

Исходные данные

Всего существует 15 уникальных предметов ЕГЭ (русский язык, математика, физика, химия, биология, история, обществознание, литература, информатика, география, английский язык, немецкий язык, французский язык, испанский язык, китайский язык).

В среднем на одно направление требуется $K_subjects = 3$ предмета.

Размер одного элемента массива `required_subjects` в текущей модели:
`subject(50) + minimum_points(4) = 54 bytes`.

Расчет объема для коллекции subjects

В коллекции subjects будет храниться 15 документов:

$$\text{sizeSubjects_collection} = 15 * (_id(12) + \text{name}(50)) = 15 * 62 = 930 \text{ bytes}$$

Расчет объема для одного документа программы

Фактический размер массива предметов:

$$\text{sizeSubjects_fact} = K_subjects * 54 = 3 * 54 = 162 \text{ bytes}$$

"Чистый" размер массива предметов (при нормализованном подходе):

$$\begin{aligned} \text{sizeSubjects_normalized} &= K_subjects * (_id(12) + \text{minimum_points}(4)) = 3 * \\ 16 &= 48 \text{ bytes (массив объектов с id предмета и баллами)} \end{aligned}$$

Разница на один документ программы:

$$\text{diffPerProgram} = 162 - 48 = 114 \text{ bytes}$$

Общая избыточность

Количество направлений можно выразить через количество вузов.

Для расчетов примем среднее значение $K_dir = 8$ направлений на один вуз (на основе анализа предметной области). Тогда:

$$P = K_dir * U = 8U$$

Для всех документов программ:

$$\text{totalRedundancy} = P * 114 = 8U * 114 = 912U \text{ (bytes)}$$

Дополнительно добавляется объем коллекции subjects (930 байт), но он не зависит от количества вузов и программ.

Коэффициент избыточности

Вычисленный итоговый объем каждой коллекции:

$\text{baseUni} = 1664$ (размер одного документа в коллекции universities)

$\text{totalProg} = 1489$ (размер одного документа в коллекции programs, с учетом $K_subjects = 3$)

$\text{baseAdmin} = 130$ (размер одного документа в коллекции admins)

Общий фактический объем модели (из предыдущего раздела):

$\text{modelSize}(U) = U * \text{baseUni} + (U * K_dir) * \text{totalProg} + A * \text{baseAdmin} =$

$= U * 1664 + U * K_dir * 1489 + 3 * 130 =$

$= (1664 + 1489K_dir) * U + 390 = 13576U + 390$

Объем нормализованной модели (с вынесенными предметами):

$\text{modelSizeNormalized}(U) = \text{modelSize}(U) - \text{totalRedundancy} + \text{sizeSubjects_collection} =$

$= (13576U + 390) - 912U + 930 = 12664U + 1320$

Коэффициент избыточности:

$R(U) = \text{modelSize}(U) / \text{modelSizeNormalized}(U) = (13576U + 390) / (12664U + 1320)$

Примеры расчета

Для $U = 100$ вузов:

$\text{modelSize}(100) = 1\,357\,990$ байт

$\text{modelSizeNormalized}(100) = 12664 * 100 + 1320 = 1\,266\,400 + 1320 = 1\,267\,720$ байт

$$R(100) = 1\,357\,990 / 1\,267\,720 \approx 1.071$$

Для $U = 1000$ вузов:

$$\text{modelSize}(1000) = 13\,576\,390 \text{ байт}$$

$$\text{modelSizeNormalized}(1000) = 12\,664\,000 + 1320 = 12\,665\,320 \text{ байт}$$

$$R(1000) = 13\,576\,390 / 12\,665\,320 \approx 1.072$$

Предельное значение

При увеличении количества вузов коэффициент избыточности стремится к:

$$\lim(U \rightarrow \infty) R(U) = 13576 / 12664 \approx 1.072$$

Вывод: избыточность модели составляет около 7.2% за счет дублирования названий предметов ЕГЭ в документах направлений подготовки. Это приемлемый показатель, учитывая, что:

1. Количество уникальных предметов невелико (всего 15);
2. Такая схема позволяет получать все данные о направлении одним запросом без дополнительных обращений к связанным коллекциям;
3. Абсолютный объем избыточных данных незначителен (около 91 КБ на 100 вузов).

Направление роста модели при увеличении количества объектов каждой сущности

При увеличении числа вузов рост будет линейным - $O(N)$. Чем больше вузов, тем больше модель. При увеличении числа направлений размер модели также будет расти пропорционально количеству вузов.

Рост при увеличении количества вузов

При фиксированном K_{dir} рост модели описывается формулой:

$$modelSize(U) = U * baseUni + (U * K_{dir}) * totalProg + A * baseAdmin =$$

$$= U * 1664 + U * K_{dir} * 1489 + 3 * 130 =$$

$$= (1664 + 1489K_{dir}) * U + 390$$

Зависимость линейная по U .

Рост при увеличении среднего количества направлений на вуз

При фиксированном количестве вузов U рост модели от K_{dir} :

$$modelSize(K_{dir}) = (1664 + 1489K_{dir}) * U + 390$$

Также линейная зависимость.

Рост при увеличении количества предметов на направление

Общая формула с учетом всех переменных:

$$modelSize(U, K_{dir}, K_{subjects}) = U * 1664 + U * K_{dir} * (1327 + K_{subjects} * 54) + 390$$

где:

1327 — базовая часть документа программы без учета предметов;

54 — размер одного предмета с минимальными баллами ($subject(50) + minimum_points(4)$).

Рост при увеличении количества администраторов

Количество администраторов константно (3) и не зависит от других сущностей. При изменении этого числа рост будет линейным, но вклад этой сущности в общий объем модели пренебрежимо мал.

Примеры данных

Пример документа коллекции universities

```
```json
{
 "_id": ObjectId("65a1b2c3d4e5f6a7b8c9d0e1"),
 "name": "Московский государственный университет им. М.В.
Ломоносова",
 "city": "Москва",
 "has_dormitory": true,
 "military_dept": true,
 "website": "msu.ru",
 "comment": "Главное здание на Воробьевых горах. День открытых
дверей 15 мая.",
 "createdAt": ISODate("2025-09-01T00:00:00Z"),
 "updatedAt": ISODate("2026-03-20T00:00:00Z")
}
```
```

Пример документа коллекции programs

```
```json
{
 "_id": ObjectId("75b1c2d3e4f5a6b7c8d9e0f2"),
 "university_id": ObjectId("65a1b2c3d4e5f6a7b8c9d0e1"),
 "name": "Прикладная математика и информатика",
 "budget_places": 25,
 "passing_score": 287,
}
```

```

"required_subjects": [
 {
 "subject": "Математика",
 "minimum_points": 75
 },
 {
 "subject": "Физика",
 "minimum_points": 70
 },
 {
 "subject": "Русский язык",
 "minimum_points": 60
 }
],
"comment": "Сильная математическая школа. Высокий конкурс.",
"createdAt": ISODate("2025-09-01T00:00:00Z"),
"updatedAt": ISODate("2026-03-20T00:00:00Z")
}
...

```

Пример документа коллекции admins

```

``json
{
 "_id": ObjectId("85c1d2e3f4a5b6c7d8e9f0a3"),
 "username": "admin",
 "password_hash":
"$2a$10$N9qo8uLOickgx2ZMRZoMy.Mr/.i/BoBdC6iN6IYfCtX1JmYx.",
 "createdAt": ISODate("2026-01-10T00:00:00Z")
}
...

```

Примеры запросов

### Пример 1. Поиск вузов по фильтрам

Запрос находит все вузы в Москве с общежитием и военной кафедрой:

```
db.universities.find({
 "city": "Москва",
 "has_dormitory": true,
 "military_dept": true
})
```

Количество запросов: 1

Задействованные коллекции: universities

Сложность:  $O(\log N)$  при наличии составного индекса по полям фильтрации

### Пример 2. Поиск направлений внутри вуза по фильтрам

Запрос находит направления в МГУ с бюджетными местами, проходным баллом не выше 280 и обязательным предметом "Математика" с минимальным баллом не более 80:

```
db.programs.find({
 "university_id": ObjectId("65a1b2c3d4e5f6a7b8c9d0e1"),
 "budget_places": { $gt: 0 },
 "passing_score": { $lte: 280 },
 "required_subjects": {
 $elemMatch: {
 "subject": "Математика",
 "minimum_points": { $lte: 80 }
 }
 }
})
```

Количество запросов: 1

Задействованные коллекции: programs

Сложность:  $O(\log P)$  при наличии индексов по полям фильтрации

Пример 3. Добавление нового вуза (для администратора)

```
db.universities.insertOne({
 "name": "Московский физико-технический институт",
 "city": "Долгопрудный",
 "has_dormitory": true,
 "military_dept": true,
 "website": "mipt.ru",
 "comment": "Ведущий технический вуз страны",
 "createdAt": new Date(),
 "updatedAt": new Date()
})
```

Количество запросов: 1

Задействованные коллекции: universities

Сложность:  $O(1)$

## **2.2. Реляционная модель**

### **Графическое представление**



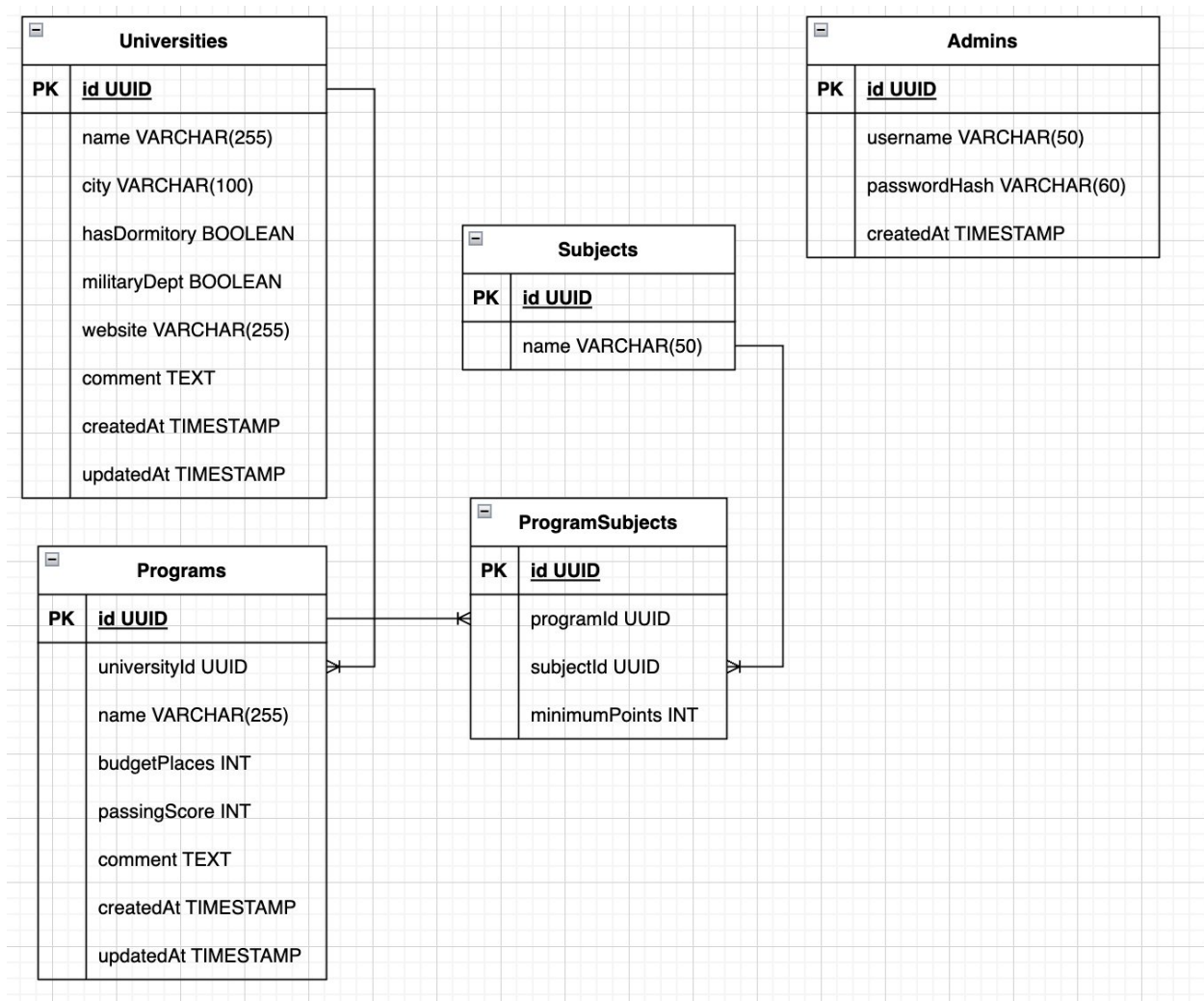


Рисунок 3 – Реляционная модель

Описание назначений коллекций, типов данных и сущностей

База данных состоит из 5 таблиц:

Universities, Programs, Subjects, ProgramSubjects, Admins.

Universities

Таблица содержит информацию о вузах.

Хранит:

- id — уникальный идентификатор (UUID, 16 байт)
- name — полное название вуза (VARCHAR(255))
- city — город (VARCHAR(100))
- hasDormitory — наличие общежития (BOOLEAN, 1 байт)

- `militaryDept` — наличие военной кафедры (BOOLEAN, 1 байт)
- `website` — ссылка на официальный сайт (VARCHAR(255))
- `comment` — комментарий (TEXT, для расчётов принят размер 1024 байта)
- `createdAt` — дата создания записи (timestamp, 8 байт)
- `updatedAt` — дата обновления записи (timestamp, 8 байт)

Размер одной записи:

$$16 + 255 + 100 + 1 + 1 + 255 + 1024 + 8 + 8 = 1668 \text{ bytes}$$

## Programs

Таблица хранит направления подготовки.

Поля:

- `id` — UUID (16 байт)
- `universityId` — ссылка на вуз (UUID, 16 байт)
- `name` — название направления (VARCHAR(255))
- `budgetPlaces` — количество бюджетных мест (INTEGER, 4 байта)
- `passingScore` — проходной балл (INTEGER, 4 байта)
- `comment` — комментарий (TEXT, принято 1024 байта)
- `createdAt` — timestamp (8 байт)
- `updatedAt` — timestamp (8 байт)

Размер записи:

$$16 + 16 + 255 + 4 + 4 + 1024 + 8 + 8 = 1335 \text{ bytes}$$

## Subjects

Таблица содержит список всех возможных предметов ЕГЭ.

Поля:

- `id` — UUID (16 байт)
- `name` — название предмета (VARCHAR(50))

Размер записи:

$$16 + 50 = 66 \text{ bytes}$$

### ProgramSubjects

Таблица связывает направления подготовки и предметы ЕГЭ.

Каждая запись означает, что для конкретной программы требуется определённый предмет.

Поля:

- id — UUID (16 байт)
- programId — ссылка на программу (UUID, 16 байт)
- subjectId — ссылка на предмет (UUID, 16 байт)
- minimumPoints — минимальный балл (INTEGER, 4 байта)

Размер записи:

$$16 + 16 + 16 + 4 = 52 \text{ bytes}$$

### Admins

Таблица содержит администраторов системы.

Поля:

- id — UUID (16 байт)
- username — логин администратора (VARCHAR(50))
- passwordHash — хэш пароля (VARCHAR(60))
- createdAt — timestamp (8 байт)

Размер записи:

$$16 + 50 + 60 + 8 = 134 \text{ bytes}$$

Оценка объёма информации, хранимой в модели

В расчетах используются следующие константы:

- $x$  — количество вузов;
- $K_{dir}$  — среднее количество направлений подготовки на один вуз;
- $K_{subjects}$  — среднее количество предметов ЕГЭ на одно направление;
- $A$  — количество администраторов.
- $K_{prog} = 8$
- $K_{subjects} = 3$
- $A = 3$

Размер таблицы Universities

Размер одной записи:

$$16 + 255 + 100 + 1 + 1 + 255 + 1024 + 16 = 1668 \text{ bytes}$$

Размер таблицы:

$$\text{UniversitiesSize}(x) = 1668x$$

Размер таблицы Programs

Количество программ:

$$\text{ProgramsCount} = x * K_{prog} = 8x$$

Размер одной записи:

$$16 + 16 + 255 + 4 + 4 + 1024 + 16 = 1335 \text{ bytes}$$

Размер таблицы:

$$\text{ProgramsSize}(x) = 1335 * 8x = 10680x$$

Размер таблицы ProgramSubjects

Количество записей:

$$\text{ProgramSubjectsCount} = x * K_{\text{prog}} * K_{\text{subjects}} = x * 8 * 3 = 24x$$

Размер записи:

$$16 + 16 + 16 + 4 = 52 \text{ bytes}$$

Размер таблицы:

$$\text{ProgramSubjectsSize}(x) = 52 * 24x = 1248x$$

Размер таблицы Subjects

Количество предметов фиксировано 15

Размер записи:

$$16 + 50 = 66 \text{ bytes}$$

Размер таблицы:

$$\text{SubjectsSize} = 66 * 15 = 990 \text{ bytes}$$

Размер таблицы Admins

Размер записи:

$$16 + 255 + 255 + 8 = 534 \text{ bytes}$$

Число администраторов принимается константой:

$$A = 3$$

Размер таблицы:

$$\text{AdminsSize} = 534 * 3 = 1602 \text{ bytes}$$

Полный размер модели

Общий объём базы данных:

$$\begin{aligned} \text{modelSize}(x) &= \text{UniversitiesSize}(x) + \text{ProgramsSize}(x) + \\ &\text{ProgramSubjectsSize}(x) + \text{SubjectsSize} + \text{AdminsSize} \end{aligned}$$

Подставим значения:

$$\text{modelSize}(x) = 1668x + 10680x + 1248x + 990 + 1602$$

Итоговая формула:

$$\text{modelSize}(x) = 13596x + 2592$$

Избыточность данных

Потенциальный источник избыточности — использование UUID в таблице ProgramSubjects.

Можно было бы использовать составной первичный ключ (programId, subjectId).

Это позволило бы убрать поле UUID (16 байт) из каждой записи.

Количество записей:

$$24x$$

Экономия памяти:

$$16 * 24x = 384x \text{ bytes}$$

Тогда размер модели без избыточности:

$$\text{modelSizePure}(x) = 13596x + 2592 - 384x$$

$$\text{modelSizePure}(x) = 13212x + 2592$$

Коэффициент избыточности:

$$\text{modelSize}(x) / \text{modelSizePure}(x)$$

При больших  $x$ :

$$\approx 1.029$$

Избыточность составляет около 2.9%.

Направление роста модели

Основной вклад в рост модели вносят таблицы:

Programs

ProgramSubjects

Таблицы Subjects и Admins имеют фиксированный размер.

Примеры данных

Universities

```
INSERT INTO Universities (
```

```
 id,
```

```
 name,
```

```
 city,
```

```
 hasDormitory,
```

```
 militaryDept,
```

```
 website,
```

```
 comment,
```

```
 createdAt,
```

```
 updatedAt
```

```
)
```

```
VALUES (
```

```
 '1234',
```

```
 'Московский государственный университет им. Ломоносова',
```

```
 'Москва',
```

```
 true,
```

```
 true,
```

```
 'https://msu.ru',
```

```
 'Московский государственный университет имени М.В. Ломоносова
```

— один из старейших и крупнейших классических университетов России',

```
 CURRENT_TIMESTAMP,
```

```
 CURRENT_TIMESTAMP
```

```
);
```

## Programs

```
INSERT INTO Programs (
 id,
 universityId,
 name,
 budgetPlaces,
 passingScore,
 comment,
 createdAt,
 updatedAt
)
VALUES (
 '4321',
 '1234',
 'Прикладная информатика',
 120,
 285,
 'Подготовка специалистов в области IT',
 CURRENT_TIMESTAMP,
 CURRENT_TIMESTAMP
);
```

## Subjects

```
INSERT INTO Subjects (id, name)
```



```
VALUES ('1', 'Математика');
```

Admins

```
INSERT INTO Admins (id, email, passwordHash, createdAt)
VALUES (
 '9999',
 'admin@admin.ru',
 'AB234FDDFG324',
 CURRENT_TIMESTAMP
);
```

Примеры запросов

Просмотр направлений университета

```
SELECT
 name,
 budgetPlaces,
 passingScore
FROM Programs
WHERE universityId = '1234';
```

Просмотр предметов для программы

```
SELECT
 s.name,
 ps.minimumPoints
```

```
FROM ProgramSubjects ps
JOIN Subjects s ON ps.subjectId = s.id
WHERE ps.programId = '4321';
```

Полная информация о программе

```
SELECT
 p.name,
 p.passingScore,
 u.name AS university,
 s.name AS subject,
 ps.minimumPoints
FROM Programs p
JOIN Universities u ON p.universityId = u.id
JOIN ProgramSubjects ps ON ps.programId = p.id
JOIN Subjects s ON s.id = ps.subjectId
WHERE p.id = '4321';
```

Запрос использует JOIN нескольких таблиц, но при наличии индексов работает эффективно.

### **2.3. Сравнение моделей**

MongoDB:

- Все данные о программе (название, бюджетные места, проходной балл, список предметов) хранятся в одном документе
- Для получения полной страницы направления нужен 1 запрос
- Для получения страницы вуза со списком направлений — 2 запроса

- Для сложного поиска по фильтрам — 1 запрос

Плюсы: минимальное количество запросов, все данные сразу доступны, простота разработки

Минусы: избыточность ~7.2%

SQL:

- Данные о программе и предметах хранятся в разных таблицах
- Для получения полной страницы направления с предметами нужен 1 запрос с JOIN (объединение 3-4 таблиц)
- Для получения страницы вуза со списком направлений — 1 запрос с JOIN
- Данные нормализованы, каждый факт хранится в одном месте

Плюсы: консистентность данных, отсутствие дублирования, мощный язык запросов

Минусы: сложные JOIN'ы могут быть медленными на больших объемах, более сложные запросы

Количество запросов для основных сценариев

Сценарий    MongoDB    SQL

Получение страницы вуза с направлениями    2 запроса    1 запрос (с JOIN)

Получение полной информации о программе    1 запрос    1 запрос (с JOIN 3-4 таблиц)

Поиск по фильтрам    1 запрос    1 запрос (с JOIN)

Добавление новой программы    1 запрос    3-4 запроса (вставка в несколько таблиц)

Вывод: для решения поставленной задачи (каталог вузов для абитуриентов) обе модели являются рабочими, но с небольшим преимуществом в сторону MongoDB:

1. Характер нагрузки: Основная операция в приложении — чтение данных (поиск и просмотр вузов/программ). MongoDB позволяет получать все данные за 1-2 запроса без сложных JOIN'ов.

2. Простота разработки: Отсутствие жесткой схемы позволяет быстро добавлять новые поля и изменять структуру данных без миграций. В SQL изменение структуры потребует ALTER TABLE и обновления нескольких связанных таблиц.

3. Соответствие структуре данных: Информация о программе (включая список предметов ЕГЭ) естественным образом представляется в виде вложенного документа, а не набора связанных таблиц.

4. Операции записи: При добавлении новой программы в MongoDB требуется 1 запрос, в SQL — 3-4 запроса (вставка в программы, вставка связей с предметами).

### 3. РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ

#### 3.1. Краткое описание и технологический стек

Было реализовано полнофункциональное веб-приложение для управления базой данных каталога ВУЗов.

Система позволяет:

- Вести каталог ВУЗов, направлений
- Выполнять сложный поиск с фильтрацией
- Импортировать/экспортировать данные
- Добавлять ВУЗы, направления в ВУЗах.

Технологический стек:

Frontend — React

Backend — Python + FastApi

DB — MongoDB

Клиентская часть реализована в виде приложения на базе React.

Она включает в себя следующие модули:

- Каталог ВУЗов — отображение списка вузов с возможностью посмотреть все направления для ВУЗа.
- Страница ВУЗа — подробная информация о вузе, год основания, количество студентов, факультетов, направлений, ссылка на сайт, адрес, есть ли военная кафедра, общежитие.
- Форма поиска — фильтрация с множественным выбором критериев.
- Можно фильтровать по частичному названию, городу, наличию общежития, военной кафедры, рейтингу, количеству направлений для ВУЗов. Направления в ВУЗе можно фильтровать по количеству

бюджетных и платных мест, проходному баллу, форме обучения, предметам ЕГЭ.

- Панель администратора — позволяет создавать и редактировать информацию о ВУЗах и направлениях, добавлять вузы и направления, импортировать/экспортировать данные.

Бэкенд предоставляет REST API, реализующий:

- Аутентификация администраторов — вход по логину и паролю с получением ID для последующих запросов.
- CRUD для университетов — создание, чтение, обновление и удаление записей о вузах.
- CRUD для образовательных программ — создание, чтение, обновление и удаление направлений подготовки.
- Фильтрация университетов — по названию, городу, наличию общежития/военной кафедры, рейтингу и количеству программ.
- Фильтрация программ — по вузу, названию, бюджетным/платным местам, проходному баллу, форме обучения и требуемым предметам.
- Поиск университетов по префиксу названия.
- Экспорт всех данных в форматах JSON, CSV (ZIP-архив) и XML.
- Импорт полного снимка данных из JSON с проверкой связей и автоматической заменой всей базы.

### **3.2. Снимки экрана приложения**

## Поиск вузов

Название вуза

📍 Город: начните вводить

🌟 Рейтинг: любой

🏠 Общежитие: не важно

🎖 Военная кафедра: не важно

📖 Направлений: любое количество

Сбросить

Применить

🔥 Найдено вузов: 7

Рейтинг: по убыванию

<b>МГУ им. Ломоносова</b>	★ 4.9	Москва	общежитие ✓	🎖 военная кафедра	3 направлений
<b>МФТИ (Физтех)</b>	★ 4.8	Долгопрудный	общежитие ✓	🎖 военная кафедра	3 направлений
<b>НИУ ВШЭ</b>	★ 4.7	Москва	общежитие ✓		3 направлений
<b>СПбГУ</b>	★ 4.7	Санкт-Петербург	общежитие ✓		3 направлений

Рисунок 4 — главная страница приложения

## Поиск вузов

Название вуза

📍 Город: начните вводить

🌟 Рейтинг: любой

🏠 Общежитие: не важно

🎖 Военная кафедра: не важно

📖 Направлений: любое количество

Сбросить

Применить

🔥 Найдено вузов: 7

Рейтинг: по убыванию

<b>МГУ им. Ломоносова</b>	★ 4.9	Москва	общежитие ✓	🎖 военная кафедра	3 направлений
<b>МФТИ (Физтех)</b>	★ 4.8	Долгопрудный	общежитие ✓	🎖 военная кафедра	3 направлений
<b>НИУ ВШЭ</b>	★ 4.7	Москва	общежитие ✓		3 направлений
<b>СПбГУ</b>	★ 4.7	Санкт-Петербург	общежитие ✓		3 направлений

Рисунок 5 — каталог ВУЗов

## **4. ВЫВОДЫ**

### **Достигнутые результаты**

В результате работы было реализовано веб-приложение, позволяющее осуществлять поиск ВУЗов и направлений в базе данных, просматривать информацию о них, при наличии прав администратора редактировать сущности. В приложении поддерживается массовый импорт/экспорт.

### **Недостатки и пути для улучшения полученного решения**

Добавлять и изменять информацию может только администратор, но если у ВУЗа поменяется информация или если была допущена ошибка, то нужна возможность сообщить об этом. Например, можно реализовать модерацию. Также в системе не реализована JWT авторизация и аутентификация.

### **Будущее развитие решения**

Реализация системы модерации, авторизации, системы уведомлений для пользователей, которые подписались на получение информации о выбранных ими ВУЗах.



## 5. ПРИЛОЖЕНИЕ

Документация по сборке и развертыванию приложения

Необходимо выполнить следующие действия:

1. Клонировать репозиторий <https://github.com/moevm/nsql1h26-uni>
2. Зайти в директорию проекта
3. Из корня выполнить

`docker compose build --no-cache && docker compose up -d`

Разработанное приложение будет доступно на <http://localhost:8080>

## **6. ЛИТЕРАТУРА**

1. Документация к MongoDB: <https://www.mongodb.com/docs/manual/>